PostHog
@posthog

What nobody tells you about writing agent skills

We're officially skill-pilled. PostHog teams have published 226 skills to our internal skill store, and we have 187 SKILL.md files across 28 different products in our codebase. Everyone uses skills to both improve PostHog and their own workflows.

They're essential because agents have amnesia. Every conversation, they rediscover your codebase and workflows from scratch. They pick the wrong tool, fall for the same gotchas, and repeat old mistakes. They figure it out eventually, but only after wasting time and tokens.

This is irritating for engineers who hate repeating themselves. That's why they automate, write abstractions, and keep handy scripts around. Skills are this instinct applied to agents. Write them once and never explain to agents again.

Here's everything we've learned about writing skills that actually work, for both you and your product.

1. You need to master progressive disclosure

Agents can learn entire programming languages, figure out how to use nearly any tool you throw at them, and improve themselves in a loop, but only if they have context to do it in.

Context is usually dumped in all at once at the beginning, which limits the amount agents have to work with. Skills flip this. Instead of loading everything up front, they disclose information progressively. The skill's information is only loaded when

the agent decides it's relevant, whether that is the main SKILL.md file or references and scripts within it. This means **good skills act as a router**.

First, the agent needs to choose to load the skill. The skill's name and description should describe when to reach for it, not what the skill is, because they're only part always in context.

Here's our SQL skill as an example:

```markdown
---
name: querying-posthog-data
description: 'Required reading before writing any HogQL/SQL or ca]
---
```

- The name and first sentence define when to read the skill.
- The middle sentence defines when to use it.
- The last sentence defines what the skill does.

Having too many skills or too large descriptions causes problems. We learned this the hard way when building (too many) skills for PostHog AI. The agent's effectiveness declined as skill descriptions filled the agent's context window. Databricks also found that agents increasingly pick the wrong skill as more are added.

Sections *within the skill* also need to work as routers. The rest of our SQL skill is a thin workflow that links to 26 schema files, 22 example query patterns, and a function index. This enables further progressive disclosure.

If done correctly, skills help agents succeed faster with fewer tokens, but only because the context isn't loaded all at once.

> **Try this:** Read your skills through the lens of progressive disclosure. Do the names and descriptions help them get discovered? Are they split in a way that agents can only load what they need?

2. Skills aren't just code

Look at these two mini-skills:

> Run git log --oneline -20.
> If you see a commit with "fix" in it, check the diff.
> If the diff touches auth.py, read lines 40–120.
> If the function is over 50 lines, flag it.

And

> Find recent changes that look risky and explain why. Start with the commit history.

They have the same spirit, but the first is over-specified. It breaks the moment the repo doesn't match what the skill expects. The second survives because it embraces ambiguity, the agent can actually look at what's in front of it.

This is how treating skills like code goes wrong. Skills are a superset of code. They can do everything code can (because agents can write code) as well as a lot of things it can't (because they are smart). Agents can handle cases in your skills you can't foresee, like errors or runtime context, but only if you let them do so.

This means skills should be precise about:

1. **The goal.** What does "done" look like? If you don't say, the agent will decide for you. Give it examples of what good looks like or ways to self-verify. For example, the creating-an-endpoint skill asks the agent to "confirm by calling endpoint-run with a sample payload to verify the response shape."
2. **Constraints.** The boundaries and guardrails that prevent disaster.
3. **Context it can't derive.** Where your data lives, which tool to use, the schema it would otherwise need to guess.

But ambiguous about:

1. **The steps.** You don't need to be prescriptive about what files to check. Tell it to understand how billing works and it will find them.
2. **Failures.** You can't know everything that will go wrong. The agent can read the error and you're paying it to figure it out.
3. **Runtime specifics.** What the data actually looks like. Line numbers, file lists, counts, versions, and outputs all change more often than you think.

Over-specification turns a skill into a workflow and strips the intelligence you are paying for. Leaving the path ambiguous gives agents the flexibility they need to do what you want them to do.

> **Try this:** Make your skills *less* prescriptive. Give the agent the tools they need and the goal you want them to achieve then let them find a way to do it.

3. Skills rot (but you can prevent it)

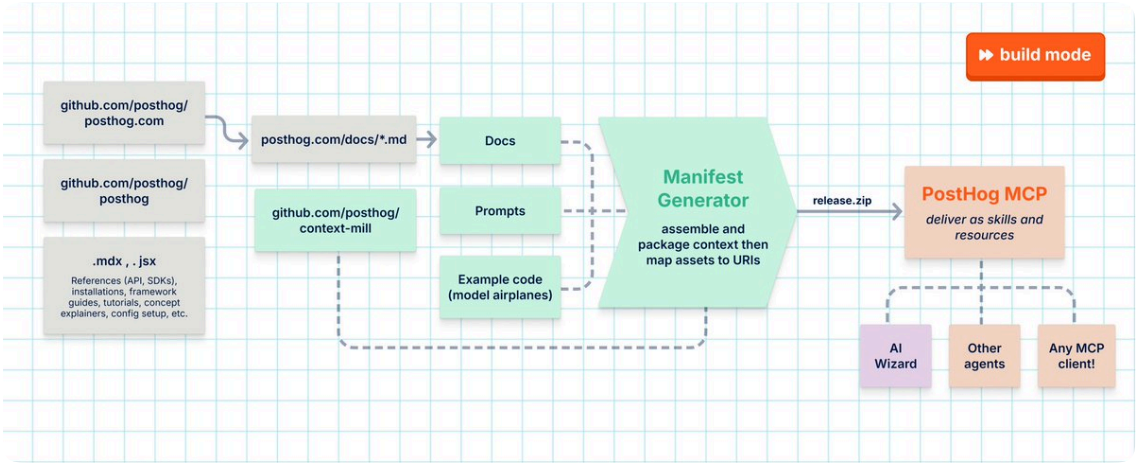
Products, APIs, CLIs, MCPs, models, behaviors, and best practices all change. When these changes break skills, most people just make fixes ad hoc, but this wastes time and tokens whenever it happens.

You can be more proactive than that. Preventing rot comes down to three principles:

1. **Split durable structure from volatile content.** Hand-write parts that rarely change, *reference* parts that drift.

2. **Point to a single source of truth.** For us, that's docs. Skills that point to specific URLs are less likely to go out of date than ones that include all the content separately.
3. **Regenerate, don't patch.** Rewrite and refine your skill from a stable base rather than piling on fixes. Repeated fixes cloud a skill's focus until it rots into something less effective.

These principles work at the individual level, but can also scale to a robust pipeline for skill generation. Our Wizard & Docs team do this to publish 120+ skills on how to integrate and use PostHog correctly.



The main part of this pipeline is the context mill (above) which consists of three parts:

1. **Sourcing:** Pulls all our docs plus curated prompts and working example apps.
2. **Assembly:** Transforms and packages these into a portable, self-contained zip manifest.
3. **Delivery:** Creates a versioned release that is used by both the PostHog MCP (as resources and slash commands) and the install wizard.

You don't need a pipeline like this to prevent your skills from rotting. The three principles work by hand. Done right, a skill doesn't just resist rot, it improves on its own. Point it at the docs and it tracks their changes. Regenerate it from a prompt and it inherits model and implementation upgrades.

> **Try this:** Next time you need to do an ad hoc fix for a skill, think of and implement a way to prevent future rot like pointing to a URL or generating a part of the skill instead.

4. Asking questions > making demands

Agents, like people, work better when you understand them and what they need. So rather than jumping straight to making demands, try asking questions like:

1. **"What can you do to help?"** You need to know what tools and capabilities an agent has to help you accomplish your task. You should care about agent

ergonomics and let it shape the skills you write.

2. **“What do you need to do better?”** Beyond what’s built-in, there might be other tools, capabilities, context access, or evaluations that might help it improve the skill.
3. **“Based on the last run, how can this skill improve?”** Runs often uncover errors or inefficiencies that can be fixed by tweaking the skill.

This works because the agent holds information you don’t: which tools it has, the context it can reach, and what broke on previous runs. Guessing that is how you burn tokens and time; asking is how you stop. As Charles said “The single most useful hour I spent with Cowork was asking it to help me figure out what to use Cowork for.”

> **Try this:** Before building your next skill, build some empathy for the agent by asking the questions above. This is high leverage work, so use a more powerful model, like Fable, even if a less powerful model will run the skill.

5. Not everything deserves a skill

Reading all this might inspire you to write skills for everything, but remember there are context and maintenance costs for each one. Picking the right ones matters, so you should write skills for work that...

- **You do repeatedly.** Point your agent at your last 30 days of work. Your Slack, emails, code, writing. Have it identify manual workflows worth packaging. Kristopher Dunham suggests asking “Have you done it three times and will you do it three more times?”
- **Agents do badly by default.** Ask your agent to do the skill you’re planning to write. If it can accomplish it efficiently, it doesn’t need a skill. For example, it doesn’t really need a skill to write SQL, but it might need a skill to write ClickHouse-flavored SQL specific to PostHog.
- **Needs context models don’t have out-of-the-box.** For example, a bunch of data like AI observability and logs in PostHog have nested or non-obvious payload shapes. A skill ensures agents don’t need to rediscover this shape every conversation.
- **Can run on autopilot.** You can create loops to handle this. Add a goal, context, way to evaluate work, and put it on a schedule. Examples include PR babysitting, flaky test hunting, and performance improvement autoresearching. Our scouts are long-running agents guided by a skill.

> **Try this:** Run this prompt to create some new skill candidates.

markdown

```
Look over my work from the last 30 days and identify repeated manu

Use available evidence in this order:
- Recent sessions and task summaries.
```

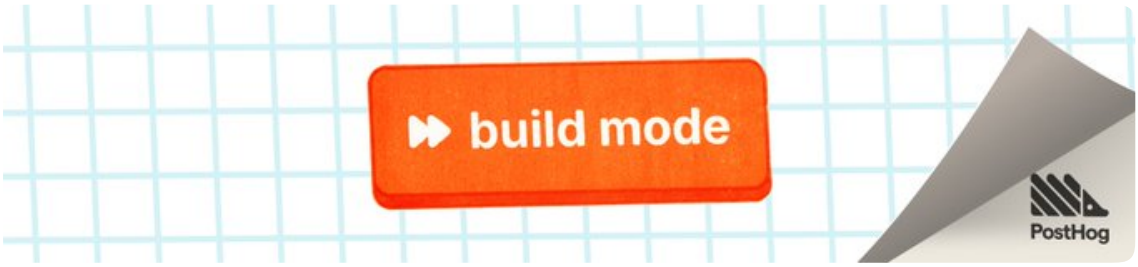
```
- Memories and outputs to find patterns repeated across sessions.
- Existing skills, custom agents, and automations, so you reuse or

Look for work that is repeated, time-consuming, error-prone, conte

Only act on a candidate when it:
- Occurred at least twice, or is clearly likely to recur and cost]
- Has stable inputs, a repeatable procedure, and a clear output or
- Would materially improve speed, quality, consistency, or reliabi
- Is not already adequately covered.

Produce a compact shortlist with:
- Repeated workflow
- Supporting evidence and dates
- Frequency/confidence
- Why it is or is not worth creating
```

Words by Ian Vanagas, who's definitely skill-pilled. *Originally published in **build mode**, a free newsletter by PostHog.*



21:29 · 3 авг. 2026 г. · **25,7 тыс.** Просмотры

10 12 297 625

AI Mastery Guide @aiseomastery · 4 авг.
226 skills across their own products is a solid signal it actually works internally.

1 208

Sotiris Gianniris @sotirisgiann · 4 авг.
absolute killer post

1 137

Mikhail @mikhailmits · 3 авг.
great writing